

Llibre electrònic

Lloc: AULES
Curs: Programació
Llibre: Llibre electrònic

Imprés
per: ALEJANDRO GONZÁLVEZ TEIJEIRO
Data: divendres, 18 de setembre 2026,
16:09



Taula de continguts

1. Introducció

2. Variables

2.1. Àmbit de les variables

3. Literals

4. Operadors

4.1. Operadors aritmètics

4.2. Operadors relacionals

4.3. Operadors lògics

4.4. Operadors de bits

4.5. Operadors d'assignació

4.6. Precedència d'operadors

4.7. Càsting d'expressions

5. Constants

6. Tipus enumerats

7. Comentaris

8. Entrada i eixida

8.1. Eixida per pantalla

8.2. Entrada de dades per teclat



1. Introducció

Com a repàs de la unitat anterior, cal destacar la manera en la que anem a crear les nostres aplicacions Java™: com a punt de partida elaborarem un fitxer de text pla amb extensió “.java”, i el fitxer rebrà el nom de l'aplicació que vulguem crear. A dins del fitxer apareixerà el que es veu a la *figura 1*:

```
public class hola {  
    public static void main (String[] args) {  
        System.out.println ("HoLa món");  
    }  
}
```

Figura 1: hola.java, la nostra primera aplicació

Cal tenir en compte que el fitxer i el nom que apareix després de “`public static void`” han de coincidir exactament. Finalment compilarem amb el comandament “`javac`” i executarem amb “`java`”.



2. Variables

Les variables poden ser considerades com a espais de memòria destinats a emmagatzemar informació per al seu ús posterior. Les variables disposen d'un tipus que determina quina informació anem a poder desar en elles. Els tipus bàsics de variables en JavaTM són els següents:

- byte, short, int i long: Es fan servir per tal d'emmagatzemar dades de tipus enter, és a dir, números positius o negatius. La diferència entre estos tipus radica en el rang de representació de que disposa cadascun, ja que fan servir 8, 16, 32 i 64 bits respectivament. Les variables d'aquesta llista també poden dur els modificadors de tipus "signed" o "unsigned" per tal d'indicar si els numeros van a disposar o no de signe.
- float i double: En aquest cas es tracta de tipus de variables destinades a mantenir números fraccionaris, és a dir, números amb decimals amb un rang de representació elevat.
- boolean: Aquest tipus de variable s'usa per a guardar valors lògics, que poden ser "vertader" o "fals".
- char: Les variables d'aquest tipus s'usen per a emmagatzemar lletres, però únicament una per variable, és a dir, no es poden emmagatzemar paraules senceres. Cal tindre en compte que una variable d'aquesta classe emmagatzema internament el codi numèric del caràcter, pel que podem realitzar amb ella operacions pròpies dels enters sense cap problema.

Tot seguit (*figura 2*) tens una taula amb l'espai que ocupa cada tipus de variable, juntament amb el seu rang de representació:

Tipus	Grandaria en bits	Valor mínim	Valor màxim
byte	8	-128	127
unsigned byte	8	0	255
short	16	-32.768	32.767
unsigned short	16	0	65.535
int	32	-2.147.483.648	2.147.483.647
unsigned int	32	0	4.294.967.295
long	64	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
unsigned long	64	0	18.446.744.073.709.551.615
float	32	$-3,4 \cdot 10^{38}$	$3,4 \cdot 10^{38}$
double	64	$-1,7 \cdot 10^{308}$	$1,7 \cdot 10^{308}$

Figura 2: Rang de representació de les diferents variables en Java

Per tal fer ús d'una variable en una aplicació JavaTM, primer cal declarar-la, i per a fer-ho cal indicar primer el tipus de la mateixa i després donar-li un nom, per exemple: si vull desar l'edat d'una persona en una variable de tipus "int" o enter, escriurem: "int edat;". És important no oblidar-nos del ";" final, ja que la majoria de sentències en JavaTM van a portar-lo.

Quan es declara una variable cal assignar-li un nom o identificador i aquest ha de seguir una regles per a ser correcte des d'un punt de vista sintàctic, les regles són les següents:

- Les variables han de començar per lletra, el caràcter "_" o el símbol del dòlar "\$".
- Després del primer caràcter podem fer servir números, però mai espais en blanc ni els símbols dels operadors.
- No poden coincidir amb cap paraula reservada del llenguatge JavaTM.
- El caràcter "ç", les lletres accentuades i la "ñ" es poden fer servir, encara que el seu ús no es troba recomanat.
- Cal tindre en compte que JavaTM és sensible a les majúscules.

Un altre tipus de dada que cal tindre en compte encara que no siga un tipus bàsic és "String" i permet emmagatzemar cadenes de caràcters que qualsevol mida. Aquest tipus de 'variable' anem a fer-lo servir de manera habitual i és per això que l'introduïm abans del moment en el seria adequat fer-ho.



2.1. Àmbit de les variables

Una variable té un àmbit d'ús, i aquest ve determinat pel bloc de codi on es troba declarada. Les variables que anem a fer servir a la present unitat es declaren al cos d'un mètode i, possiblement, a l'interior d'un bloc d'instruccions donat i serà allà on van a poder fer-se servir. Fer referència a una variable fora del seu àmbit d'ús no és possible i comportarà un error en temps de compilació de la nostra aplicació.

Les variables deuen ser inicialitzades abans de fer-les servir per tal que tinguin un valor esperable, encara que el llenguatge Java assigna valor per defecte a les variables de la següent manera: les variables numèriques s'inicialitzen a 0, les de tipus caràcter al valor `'\u0000'` i les de tipus lògic al valor `'false'`.

A més dels tipus de variables que hem vist al primer paràgraf i que s'anomenen variables locals, a Java disposem de dos tipus de variable més i que anirem veient al llarg de les pròximes unitats, són les variables d'instància (que pertanyen a un objecte en concret) i les variables de classe (pertanyen a una classe donada i a tots els objectes de la mateixa).



3. Literals

A dins d'un programa ens pot fer falta referir-nos o indicar un valor en concret o literal (per a assignar-lo a una variable, comparar-lo...) i per a fer-ho hem de tindre en compte de quin tipus va a ser el literal; tenim literals dels mateixos tipus que les variables disponibles:

- Literals lògics: Els literals lògics sols poden ser dos, i s'escriuen directament com `"true"` o `"false"`.
- Literals enters: Si el que desitgem és indicar un valor enter i expressat en base 10, procedirem a escriure el número sense cap afegit, per exemple: `"1024"`. Per contra si el número es troba en base 8 o octal, llavors el número cal que aparega precedit d'un 0, en el cas anterior, el 1024 quedaria expressat també com a `"02000"`. Si en un altre cas, els literals enters es troben en notació hexadecimal, apareixeran precedits de `"0x"`, el numero 1024 quedarà com a `"0x400"`. Quan ens trobem en el cas que ens faça falta expressar un número extremadament gran (de més de 32 bits) el numero apareixerà amb una `"L"` final, com en el cas de `"604800000000000L"` (número de nanosegons que hi ha en una setmana).
- Literals de tipus real: Per tal d'expressar números amb decimals, tenim dues opcions: la primera és la manera clàssica on separem la part entera de la fraccionària amb un punt, com pot ser en `"12.34"` o en `"-14.08"`, l'altra és fer servir la notació científica per tal d'expressar els números com a potències de 10, com és `"4.5e3"` per a indicar 4500. A més a més, també podem indicar de quin tipus exacte són els literals de tipus real, ja que estos poden ser float (de 32 bits) o double (de 64 bits). Per a indicar que un literal real és float, el farem acabar en una `"f"` final, mentre que si volem que siga de tipus doble, deixarem el número tal qual, o bé posarem una `"d"`.
- Literals de tipus caràcter: Els literals de tipus caràcter s'expressen entre cometes simples com `"'e'"` o `"'H'"`. És important destacar que les cometes simples sols s'usen per a aquesta tasca i no s'han de confondre amb les dobles que vorem a continuació.
- Literals de tipus String: Els literals d'aquest tipus no es corresponen a cap variable de les que hem vist, sinó que s'usen per a donar valors a objectes (ja vorem que són a les següents unitats) de tipus String. Els literals String s'usen per tal d'especificar cadenes de caràcters de longitud variable i s'expressen sempre entre cometes dobles com és el cas de `"'Hola a tots!'"` o de `"'e'"` (aquesta última ja no és un caràcter per anar entre cometes dobles i no simples).



4. Operadors

Ara que ja sabem que tenim variables que podem usar per a emmagatzemar informació i que també tenim literals per tal d'indicar valors de manera directa, sols els falta conèixer les formes que tenim per tal d'operar amb aquestes dades i fer que l'ordinador faci coses útils amb variables i literals. Els operadors es classifiquen en diversos tipus i no tots els tipus d'operadors es poden usar amb tots els tipus de variables, ja que hi ha operacions que no tenen sentit sobre un tipus de dades donat.

Els tipus d'operadors disponibles són: aritmètics, relacionals, lògics, de manipulació de bits i d'assignació.



4.1. Operadors aritmètics

Els operadors aritmètics són aquells destinats a treballar amb dades numèriques, per tant els anem a usar sobre variables enteres (byte, short, int o long) o sobre variables reals (float o double) i tindrem en compte que una operació aritmètica entre un enter i un real sempre donarà com a resultat un real.

Les operacions que realitzen aquests operadors són: la suma, la resta, la multiplicació i la divisió (i el seu residu). Ací tens la llista:

- Suma: L'operador de suma "+", es fa servir per tal de sumar dues variables o literals de tipus enter o reals. Aquest operador es fa servir també per tal de concatenar cadenes de caràcters.
- Resta: Este operador "-", es fa servir, bé per tal de restar dos variables o literals, bé per tal de canviar el signe.
- Multiplicació: La multiplicació "*", com podem suposar, té la finalitat d'obtenir el producte de dos números.
- Divisió: L'operador "/", permet calcular la divisió de dos números entre sí. Cal tenir en compte que en el cas que la divisió no tinga un resultat exacte, si ens trobem operant amb números reals obtindrem un número amb decimals com a resultat, mentre que si estem usant números enters els decimals del resultat seran truncats sense cap tipus d'arrodoniment.
- Residu: L'operador residu "%" sols es pot usar amb números enters, i ens proporciona el valor que té el residu de la divisió.

A continuació (figura 3) tens un exemple d'ús d'aquests operadors amb comentaris d'aclariment:

```
public class Aritmetica {  
    public static void main (String[] args) {  
        System.out.println (12 + 2); //Imprimirà 14  
        System.out.println (10 * 12); //Imprimirà 100  
        System.out.println (10 / 12); //Imprimirà 0  
        System.out.println (12 % 10); //Imprimirà 2  
  
        System.out.println (5.0 + 2); //Imprimirà 7.0  
        System.out.println (5.0 * -7.0); //Imprimirà -35.0  
        System.out.println (5.0 / -7.0); //Imprimirà -0.71428...  
    }  
}
```

Figura 3: Exemple d'ús dels operadors aritmètics



4.2. Operadors relacionals

Els operadors relacionals es fan servir per tal de comparar variables, literals o expressions compostes entre sí. Aquests operadors, al igual que els anteriors, poden operar sobre enters i reals i retornen un valor de tipus lògic com a resultat. Ací els tens:

- Operador de superioritat: L'operador de superioritat ">" té la utilitat de comparar dos valors entre si per tal d'esbrinar si el primer element de la comparació és major que el segon.
- Operador d'inferioritat: Al contrari que en el cas anterior, l'operador "<" comprova si el primer element és menor que el segon.
- Operador de major o igual: L'operador de major o igual ">=" compara dos números per a vore si el primer és major o igual que el segon.
- Operador de menor o igual: L'operador de menor o igual "<=" compara dos números per a vore si el primer és menor o igual que el segon.
- Operador d'igualtat: L'operador d'igualtat "==" es pot fer servir no sols per a vore si dues variables o literals numèrics són iguals entre sí, sinó que també es pot usar per tal de comparar referències a objectes (com vorem a les següents unitats).
- Operador de desigualtat: Aquest operador "!=" compara si dues variables, literals, expressions o referències són diferents entre sí.

A la *figura 4* tens un exemple d'ús d'aquests operadors:

```
public class Relacional {  
    public static void main (String[] args) {  
        System.out.println (25 > 5); //Imprimirà true  
        System.out.println (5 <= 25); //Imprimirà false  
        System.out.println (25 == 0); //Imprimirà false  
        System.out.println (5 != 0); //Imprimirà true  
  
        System.out.println (0.0 == 0); //Imprimirà true  
    }  
}
```

Figura 4: Exemple d'ús dels operadors relacionals



4.3. Operadors lògics

Els operadors lògics o booleans tenen la utilitat de poder treballar amb dades de tipus lògic, és a dir, vertader o fals. Els operadors són els que tens tot seguit:

- Operador de conjunció: L'operador de conjunció "**&&**" s'usa per tal de comprovar si dos valors lògics es troben els dos a vertader, cas en que el resultat serà vertader, en altre cas serà fals.
- Operador de disjunció: L'operador de disjunció "**||**" s'usa per tal de comprovar si, de dos valors lògics n'hi ha algun a vertader, cas en que el resultat serà vertader, en altre cas serà fals.
- Operador de disjunció exclusiva: L'operador de disjunció exclusiva "**^**" s'usa per tal de comprovar si, de dos valors lògics, hi ha un i sols un a vertader, cas en que el resultat serà vertader, en altre cas serà fals.
- Operador de negació: La funció d'aquest operador "**!**" és la de simplement invertir el valor lògic de l'expressió a la que s'avantposa, és a dir, passar el valor vertader a fals i de fals a vertader.

Com als apartats anteriors, ací tens un exemple aclaridor (*figura 5*):

```
public class Logic {  
    public static void main (String[] args) {  
        System.out.println (true && false); //Imprimirà false  
        System.out.println (true && true); //Imprimirà true  
        System.out.println (false || true); //Imprimirà true  
        System.out.println (true ^ true); //Imprimirà false  
        System.out.println (true ^ false); //Imprimirà true  
        System.out.println (!false); //Imprimirà true  
    }  
}
```

Figura 5: Exemple d'ús dels operadors lògics



4.4. Operadors de bits

Els operadors de bits permeten la realització d'operacions lògiques sobre els bits de variables en memòria, com per exemple: desplaçaments, inversions i operacions lògiques bit a bit; per tal d'usar-los cal tenir ben present con es desa la informació a nivell de bits. Aquestes operacions no solen resultar útils en massa casos, però val la pena conèixer-les, són les següents:

- Desplaçament lògic cap a la dreta: Realitzar un desplaçament lògic cap a la dreta ">>" implica realitzar divisions per potències de 2 sense tindre en compte el signe de la variable (sobre números negatius donarà resultats estrambòtics).
- Desplaçament aritmètic cap a la dreta: El desplaçament aritmètic cap a la dreta ">>" implica dividir per potències de 2 al valor sobre el que s'aplica (en aquest cas funcionarà correctament sobre números positius i negatius).
- Desplaçament cap a l'esquerra: Quan desplace un valor cap a l'esquerra "<<", el que fem és multiplicar-lo per potències de 2 d'una manera ràpida.
- Disjuntor lògic: Aquest operador "|" realitza l'operació lògica OR bit a bit entre dues expressions.
- Conjuntor lògic: Aquest operador "&" realitza l'operació lògica AND bit a bit entre dues expressions.
- Disjuntor exclusiu: Aquest operador "^" realitza l'operació lògica XOR bit a bit entre dues expressions.
- Negació lògica: Aquest operador "~" realitza la inversió dels bits de l'expressió sobre la que s'aplica.

Ací tens un menut exemple de com s'apliquen (*figura 6*):

```
public class Bits {  
    public static void main (String[] args) {  
        System.out.println (2 << 2); //Imprimirà 2 * 22  
        System.out.println (10 >> 1); //Imprimirà 10 / 21  
        System.out.println (-1 >>> 20); //Millor deixar-ho passar  
        System.out.println (~10); //Invertim els bits de 10  
        System.out.println (3 & 2); //Ens quedem amb els bits en comú  
        System.out.println (3 | 4); //Combinem els bits  
    }  
}
```

Figura 6: Exemple d'ús dels operadors de bits

4.5. Operadors d'assignació

Amb els operadors d'assignació comencem a poder fer coses interessants amb les variables, ja que podem modificar el valor que tenen assignant-los el valor d'una expressió on poden aparèixer variables, literals o expressions compostes. L'operador d'assignació per excel·lència és "=", ja que tota la resta d'operadors es poden expressar com a una combinació d'ell amb altres.

La funcionalitat de l'operador "=" és la de modificar el valor de la variable que té a la seua part esquerra i assignar-li el valor d'allò que apareix a la part dreta; és per això que a la part esquerra de l'operador mai apareixeran literals ni expressions compostes, sols variables.

A continuació tens un llistat d'operadors d'assignació i la seua relació amb "=":

- "var1 += var2": Equivalent a "var1 = var1 + var2".
- "var1 -= var2": Equivalent a "var1 = var1 - var2".
- "var1 *= var2": Equivalent a "var1 = var1 * var2".
- "var1 /= var2": Equivalent a "var1 = var1 / var2".
- "var1 %= var2": Equivalent a "var1 = var1 % var2".
- "var1 &= var2": Equivalent a "var1 = var1 & var2".
- "var1 |= var2": Equivalent a "var1 = var1 | var2".
- "var1 ^= var2": Equivalent a "var1 = var1 ^ var2".
- "var1 >>= var2": Equivalent a "var1 = var1 >> var2".
- "var1 <<= var2": Equivalent a "var1 = var1 << var2".
- "var1 >>>= var2": Equivalent a "var1 = var1 >>> var2".
- "var1++": Equivalent a "var1 = var1 + 1" i s'executarà després d'avaluar l'expressió on es troba.
- "++var1": Equivalent a "var1 = var1 + 1" i s'executarà abans d'avaluar l'expressió on es troba.
- "var1--": Equivalent a "var1 = var1 - 1" i s'executarà després d'avaluar l'expressió on es troba.
- "--var1": Equivalent a "var1 = var1 - 1" i s'executarà abans d'avaluar l'expressió on es troba.

Com als apartats anteriors, ací tens exemples d'ús d'aquests operadors (*figura 7*):

```
public class Assignacio {  
    public static void main (String[] args) {  
        int a; //Declarem un enter  
  
        a = 5; //Ara a val 5  
        System.out.println (a); //Imprimeix 5  
  
        a = 7; //Ara a val 7  
        System.out.println (a); //Imprimeix 7  
  
        a += 2; //Ara a val 9  
        a *= 3; //I ara 27
```

```
        System.out.println (a++); //Imprimeix 27 i a passa a valdre 28

        System.out.println (--a); //a passa a valdre 27 i després
s'imprimeix
    }
}
```

Figura 7: Exemple d'ús dels operadors d'assignació

4.6. Precedència d'operadors

Ara que ja hem vist tots els operadors i el seu significat, ens trobem amb el següent problema: que passa si ens trobem amb una expressió on apareixen operadors de diferents tipus combinats com en, per exemple, " $a = 2 + 3 * 4$ "? Doncs que haurem de veure allò que s'anomena precedència d'operadors, és a dir, quins operadors s'avaluen abans de quins altres. Ací tens la llista d'operadors de més a menys precedència:

- Parèntesi.
- Operadors unaris (que operen sobre un sol element com poden ser "!" o "-").
- Multiplicació, divisió i residu.
- Suma i resta.
- Desplaçament de bits.
- Relacionals.
- Igualtat i desigualtat.
- AND a nivell de bits.
- XOR a nivell de bits.
- OR a nivell de bits.
- AND lògic.
- XOR lògic.
- OR lògic.
- Operadors d'assignació.



4.7. Càsting d'expressions

Java™ converteix automàticament expressions i variables quan ho requereix, però poden donar-se casos en els que el compilador considere que la conversió requerida va a portar pèrdua d'informació i no realitzi les conversions per precaució, o casos en els que desitgem que una expressió s'avalue com si tinguera un tipus determinat. És per tot allò anterior que el llenguatge disposa de maneres de convertir una expressió donada d'un tipus a un altre.

La manera de convertir una expressió, consisteix en indicar-ne el tipus que volem que tinga entre parèntesi i avantposar-la a la mateixa, de manera que en, per exemple, "**(int)** 4.5" el valor numèric passarà a ser interpretat com a enter i es perdrà la seua part fraccionària.

Tot seguit tens uns exemples de conversions de tipus tant implícites com explícites (*figura 8*):

```
public class Casting {
    public static void main (String[] args) {
        System.out.println (2 + 2); //Imprimirà 4
        System.out.println (2 + 2.5); //El 2 es converteix a double i
imprimeix 4.5
        System.out.println (2.5f + 2.5); //El 2.5f es converteix de float a
double i imprimeix 5.0
        System.out.println (5.0 % 2); //Error: el residu no és pot usar amb
double
        System.out.println (((int) 5.0) % 2); //Convertim 5.0 a int
explícitament i imprimeix 1
        System.out.println ((int) (2.5f + 2)); //Convertim a int
explícitament i imprimeix 4
    }
}
```

Figura 8: Conversió implícita i explícita



5. Constants

Java™ permet la definició de valors constants, és a dir, que els seus valors no van a poder ser modificats al llarg de l'execució del programa. Els valors constants es declaren fent servir la paraula reservada "final" situada abans del tipus en la declaració d'una variable, com en: "final double pi = 3.14159265359;". L'únic lloc on se li pot donar valor a una constant és a la seua declaració de la manera com ho pots vore a l'exemple anterior.



6. Tipus enumerats

Els tipus enumerats permeten definir tipus de variable personalitzats amb una sèrie de valors possibles definits en una enumeració i permeten descriure certes propietats que no s'ajusten a cap tipus definit com per exemple: dies de la setmana, mesos de l'any, estacions i similars. Tot seguit (*figura 9*) tens un exemple on es declara un tipus enumerat per als dies de la setmana, es crea una variable d'aquest tipus i se li assigna valor:

```
public class Enumerat {  
  
    public enum diaSetmana {dilluns, dimarts, dimecres, dijous, divendres,  
dissabte, diumenge};  
  
    public static void main (String[] args) {  
        diaSetmana primer = diaSetmana.dilluns;  
        diaSetmana ultim = diaSetmana.diumenge;  
        System.out.println ("El primer dia és " + primer);  
    }  
}
```

Figura 9: Definició i ús d'un tipus enumerat



7. Comentaris

A dins del codi font de qualsevol aplicació sempre es recomana introduir comentaris aclaridors sobre allò que s'està fent, d'aquesta manera qualsevol persona que es trobe amb la necessitat de modificar o estudiar el nostre codi tindrà una ajuda per tal d'entendre'l. La quantitat de comentaris present al codi font ha de ser la correcta, ja que si en posem massa, aconseguirem confondre al lector en lloc d'ajudar-lo.

Al codi font d'una aplicació Java™ tenim dues maneres d'introduir comentaris: la primera consisteix en començar-los amb `"/"` de manera que tot el text que aparega a continuació serà ignorat pel compilador, l'altra consisteix en englobar-los amb els signes `"/*` i `*/` per tal de crear comentaris de diverses línies.



8. Entrada i eixida

La programació d'aplicacions sense la capacitat de demanar dades d'entrada a l'usuari ni de mostrar informació com a eixida genera programes amb escassa utilitat pràctica, i és per això que anem a veure com realitzar operacions simples d'entrada per teclat i eixida per pantalla per tal de llegir i mostrar la informació requerida.



8.1. Eixida per pantalla

Com ja hem vist en molts exemples anteriors, la manera més ràpida de mostrar informació per pantalla és fer servir els mètodes `print` i `println` de `System.out`, mètodes que permeten mostrar cadenes de caràcters o variables per pantalla, bé sense deixar, bé deixant un retorn de línia.

Una altra manera de mostrar eixida per pantalla és el mètode `printf` de `System.out`, mètode que ens permet mostrar informació formatada de manera més precisa del que permeten els mètodes anteriors. `printf` s'invoca sempre amb una cadena de caràcters que pot contindre certs patrons de caràcters que comencen per símbol `%`, seguida de tantes expressions indicades com a paràmetres addicionals com aparicions de `%` trobem; cada aparició d'un dels patrons anteriors serà substituït de manera correlativa per una de les expressions indicades i respectant el format indicat al patró. Tot seguit tens una llista amb la codificació de certs caràcters especials i els patrons més comuns que es poden fer servir (*figura 10*):

Caràcter	Conversió	Descripció
<code>\n</code>	retorn de carro	Mostra un salt de línia a l'eixida
<code>\t</code>	tabulador	Mostra un tabulador a l'eixida
<code>%b</code> o <code>%B</code>	boolean	Mostra un valor lògic en majúscules o minúscules segons el caràcter indicat
<code>%c</code> o <code>%C</code>	char	Mostra un caràcter Unicode en majúscula o minúscula segons el caràcter indicat
<code>%s</code> o <code>%S</code>	string	Mostra una cadena caràcters en majúscula o minúscula segons el caràcter indicat
<code>%d</code>	int	Mostra un enter en representació decimal
<code>%u</code>	unsigned int	Mostra un enter sense signe en representació decimal
<code>%h</code> o <code>%H</code>	unsigned int	Mostra un enter sense signe en representació hexadecimal; els símbols de la A a la F es mostren en majúscula o minúscula segons el caràcter especificat
<code>%o</code>	int	Mostra un enter en representació octal
<code>%x</code> o <code>%X</code>	int	Mostra un enter en representació hexadecimal; els símbols de la A a la F es mostren en majúscula o minúscula segons el caràcter especificat
<code>%f</code>	float, double	Mostra un número en coma flotant

Figura 10: Patrons existents per a `printf`

Tot seguit tens uns exemples d'ús (*figura 11*):

```
public class Printf {  
    public static void main (String[] args) {  
        System.out.printf ("%d", 4); //Imprimirà 4  
        System.out.printf ("%04d", 4); //Imprimirà 0004  
        System.out.printf ("%+d %+d", 4, -4); //Imprimirà +4 -4  
        System.out.printf ("%f", 4.0); //Imprimirà 4,000000  
        System.out.printf ("%4f", 4.0); //Imprimirà 4,0000  
        System.out.printf ("%X", 1024); //Imprimeix 400  
    }  
}
```

Figura 11: Exemples d'ús de "printf"

8.2. Entrada de dades per teclat

La manera directa per tal de llegir informació pel teclat en Java™ és fer servir el mètode "`read()`" de "`System.in`" que ens retorna un byte amb la informació llegida. Com pots suposar, llegir informació complexa fent servir aquest mètode és complicat i ens obliga a la introducció de molt de codi per tal de llegir línies, números o altres i és per això que disposem de mètodes més elaborats per tal de llegir informació per teclat. Anem a veure dues de les maneres més habituals de llegir informació del teclat:

8.2.1. La classe "`BufferedReader`"

Si el que desitgem és llegir línies de text per teclat, aquesta és la classe adient ja que ens retorna les línies de caràcters d'entrada de "`System.in`" com a "`String`". A la figura N tens un exemple d'ús de la classe, però has de tenir en compte que per a fer servir "`BufferedReader`" hem de recórrer a una classe intermedia anomenada "`InputStreamReader`".

```
import java.io.*;

public class BR {
    public static void main (String[] args) {
        BufferedReader stdin = new BufferedReader (new InputStreamReader
(System.in));
        String cadena;

        System.out.print ("Escriu alguna cosa: ");
        cadena = stdin.readLine ();
        System.out.printf ("Has escrit: " + cadena);
    }
}
```

Figura 12: Exemple d'ús de "`BufferedReader`"

"`BufferedReader`" no proporciona cap forma de llegir informació que no siga una cadena de caràcters i si el que necessitem és llegir informació numèrica o de qualsevol altre tipus haurem de recórrer a altres mètodes per tal de convertir-la. La classe que tenim a continuació proporciona una alternativa més pràctica per a llegir tant cadenes de caràcters com altre tipus d'informació.

8.2.2. La classe "`Scanner`"

Aquesta classe es va introduir posteriorment a "`BufferedReader`" i ens proporciona una funcionalitat molt més elevada amb major facilitat d'ús. "`Scanner`" disposa de mètodes per tal de llegir cadenes de caràcters, però també ens proporciona la utilitat de llegir enters de diversos tipus juntament amb floats i doubles. Tot seguit tens un exemple d'ús d'aquesta classe (*figura 13*):

```
import java.util.Scanner;
```

```
public class SC {  
    public static void main (String[] args) {  
        Scanner entrada = new Scanner (System.in);  
        double preu;  
        int quantitat;  
  
        System.out.print ("Escriu el preu del producte: ");  
        preu = entrada.nextDouble ();  
        System.out.print ("Escriu la quantitat a comprar: ");  
        quantitat = entrada.nextInt ();  
        System.out.println ("Has de pagar " + (preu * quantitat));  
    }  
}
```

Figura 13: Exemple d'ús de "Scanner"